Deep Learning Project: AI-Powered Image Similarity Search and Recommendation System

Project Report

Submitted by

Name of Student(s)	University Roll No.
Sandipa Chakrabarty	11600222137
Anushka Polley	11600222135
Sreyasree Sasmal	11600222119

Conducted at

INTEL®

Department of Computer Science & Engineering,
MCKV Institute of Engineering
243, G.T. Road(N)

Liluah, Howrah - 711204

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to INTEL® for providing us with the valuable opportunity to undertake our internship and gain practical exposure in the field of Deep Learning.

We are deeply thankful to our mentor Mr. Soham Goswami, Assistant Professor, Computer Science and Engineering, MCKVIE who guided us throughout the internship period with their continuous support, valuable insights, and constructive feedback. Their encouragement and supervision played a crucial role in helping us complete the assigned tasks successfully.

We are also profoundly grateful to our Head of Department, Mr. Avijit Bose, for his constant support and encouragement, which have been a source of motivation and inspiration for our team.

Our sincere thanks go to our principal, Dr. Abhijit Lahiri, for providing a conducive learning environment and the necessary resources to carry out this project.

Finally, we extend our gratitude to all our colleagues and team members for their cooperation, teamwork, and dedication that made this internship a rewarding learning experience.

We feel privileged to have been a part of this program, which has significantly enriched our technical knowledge, professional skills, and overall understanding of real-world applications in the field of Computer Science and Engineering.

CONTENTS

1. ABSTRACT

With the exponential growth of digital images on online platforms, the demand for intelligent and efficient image retrieval systems has become increasingly significant. Traditional keyword-based search methods often fail to capture the intrinsic visual content of images, resulting in inaccurate and limited search outcomes. To overcome these challenges, this project—**“Deep Learning Project: AI-Powered Image Similarity Search and Recommendation System”**—leverages advanced deep learning techniques for visual similarity analysis.

The system employs a **ResNet-based convolutional neural network (CNN)** architecture to extract high-level feature embeddings from images in the **Caltech101 dataset**. These embeddings are compared using similarity metrics such as **cosine similarity** to identify and retrieve visually related images. By mapping images into a semantic feature space, the system enables accurate, content-based image retrieval and recommendation without relying on manual labels or metadata.

The experimental results on the Caltech101 dataset demonstrate the capability of deep learning models in capturing semantic visual relationships and improving retrieval accuracy. The study highlights the potential of deep convolutional networks in building scalable and intelligent image retrieval systems applicable to domains such as digital libraries, visual search engines, and content-based recommendation systems.

2. INTRODUCTION

In recent years, the volume of digital images produced and shared across online platforms—including social media, e-commerce portals, and digital libraries—has grown exponentially. While the availability of image data presents opportunities, it also poses significant challenges for retrieval and organization. Traditional search systems that rely on manually assigned keywords, tags or metadata often struggle to understand the underlying visual content of images, leading to poor search precision and recall.

To overcome these limitations, content-based image retrieval (CBIR) systems have gained traction. Rather than depending solely on text annotations, CBIR systems analyse the visual features of the image itself—such as colour, texture or shape—to enable more meaningful comparison and search. With the advent of deep convolutional neural networks (CNNs), feature extraction from images has reached a new level of sophistication. Architectures such as ResNet are capable of learning high-level semantic representations that capture intrinsic visual patterns far beyond simple low-level features.

In this project titled **"Deep Learning Project: AI-Powered Image Similarity Search and Recommendation System"**, we utilise a ResNet-based CNN to extract embeddings from images sampled from the Caltech101 dataset. These embeddings represent each image in a high-dimensional feature space. Similarity between images is then computed using distance metrics such as cosine similarity. By ranking images according to their proximity in the embedding space, our system is able to retrieve visually similar items and provide recommendations. The use of a benchmark dataset like Caltech101 enables objective evaluation of the model's capability in capturing semantic similarity.

The motivation for this work lies in its applicability to modern use-cases: for example, e-commerce platforms where customers upload or view product images and desire visually related alternatives; digital libraries where users may wish to explore visually similar photographs; or content-based recommendation engines where the visual dimension plays a critical role. Through this system, we aim to demonstrate how deep learning techniques can bridge the semantic gap between human perception and machine-perceived image similarity, achieving more accurate and scalable search and recommendation functionality.

3. SCOPE OF THE WORK

The scope of this project involves the development and evaluation of an **AI-powered Image Similarity Search and Recommendation System** based on **deep learning**. The major components of the work are described below:

1. Feature Extraction using ResNet

The project utilizes a **ResNet-based Convolutional Neural Network (CNN)** to extract deep visual features from input images. The model generates high-level feature representations (embeddings) that capture semantic details such as shape, color, and texture.

2. Generation of Image Embeddings

Each image in the dataset is converted into a numerical embedding vector using the trained ResNet model. These embeddings provide a compact and discriminative representation of visual content for efficient comparison.

3. Similarity Measurement

Image similarity is measured using **cosine similarity**, which quantifies the closeness between embedding vectors. Based on these similarity scores, the system retrieves and ranks visually related images.

4. Dataset and Evaluation

The system is developed and tested on the **Caltech101 dataset**, which contains diverse object categories. This dataset serves as a benchmark to evaluate the model's ability to generalize and accurately capture visual similarity.

5. User Interface and Visualization

A simple **Python-based interactive interface** is implemented to visualize similarity results. Users can input an image and view the top retrieved similar images along with their similarity scores.

6. Limitations and Boundaries

The scope of the project is limited to **image-level similarity search** within a fixed dataset. It does not extend to text-to-image retrieval, large-scale deployment, or fine-tuning on custom domain-specific datasets.

4. SURVEY OF THE PREVIOUS WORK

1. Deep Learning for Image Retrieval (2015): [1]

Proposed the use of deep convolutional neural networks (CNNs) for extracting high-level semantic features from images to improve retrieval accuracy. Compared CNN-based feature extraction with traditional handcrafted descriptors like SIFT and SURF, demonstrating a significant improvement in identifying visually similar images.

2. Very Deep Convolutional Networks for Large-Scale Image Recognition (2015): [2]

Introduced the VGGNet architecture, which emphasized the importance of depth in CNNs for feature learning. Demonstrated that deeper convolutional layers could capture hierarchical image representations, greatly enhancing visual similarity detection and classification accuracy.

3. Deep Residual Learning for Image Recognition (2016): [3]

Proposed the ResNet (Residual Network) architecture that introduced skip connections, enabling the training of very deep networks without vanishing gradients. This innovation allowed better generalization in feature learning, making ResNet an ideal choice for visual similarity and image recommendation tasks.

4. Image Retrieval Using Deep Features and Distance Metrics (2018): [4]

Utilized deep feature embeddings extracted from pre-trained CNNs and applied distance metrics like cosine similarity and Euclidean distance to measure visual similarity between images. Highlighted that cosine similarity effectively handles high-dimensional feature spaces in retrieval systems.

5. Deep Learning-Based Image Recommendation Systems (2020): [5]

Explored integrating CNN feature extraction with recommendation algorithms to improve user experience in domains like e-commerce and multimedia platforms. The study emphasized using image embeddings to suggest visually similar or complementary items based on user preferences.

5. HARDWARE AND SOFTWARE DETAILS

5.1 Hardware Details

Processor: Intel Core i5

RAM: 8 GB

GPU: NVIDIA GeForce RTX 3050

CUDA Version: 12.7

NVIDIA Driver Version: 566.07

5.2 Software Details

Development Environment: Visual Studio Code (VS Code)

Programming Language: Python 3.x

Deep Learning Framework: PyTorch

Additional Libraries Used: torch, torchvision, numpy, matplotlib,PIL (Python Imaging Library), scikit-learn

Visualization & Debugging Tools: Jupyter extension integrated within VS Code

5.3 Datasets to be used

Name of the Dataset: Caltech101 Dataset

Source: California Institute of Technology (Caltech)
URL : https://www.kaggle.com/datasets/imbikramsaha/caltech-101

Description:
The Caltech101 dataset is a widely used benchmark dataset for object recognition and image classification tasks. It contains images from 101 object categories along with one background category. Each category includes between 40 to 800 images, with most categories containing around 50 images. The images cover a diverse range of real-world objects, such as animals, vehicles, musical instruments, and everyday items.

6. PROCEDURE

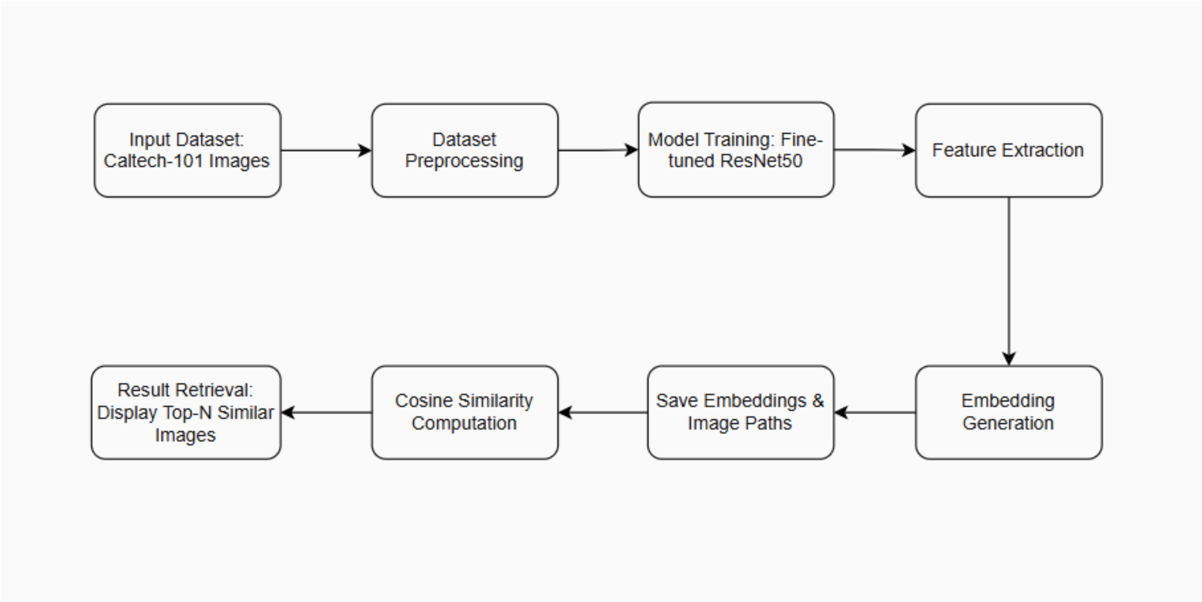

Fig: System Architecture of the Proposed Image Similarity Search Model

6.1 Dataset preparation and preprocessing

Purpose: These preprocessing steps ensure that all input images maintain a uniform format and scale, reducing variability and improving the efficiency of feature extraction.

Steps:

- **Image Resizing:**
  All images were resized to 224 × 224 pixels to match the input dimensions required by the ResNet architecture.
- **Normalization:**
  Each image was normalized using the ImageNet mean and standard deviation values:
  - Mean = [0.485, 0.456, 0.406]
  - Standard Deviation = [0.229, 0.224, 0.225]
- This normalization ensures compatibility with the pre-trained ResNet model and stabilizes feature learning.

- **Data Splitting:**
  The dataset was split into training (80%) and validation (20%) subsets to evaluate the model's generalization capability.
- **Batch Processing:**
  Data was loaded using PyTorch's DataLoader with a batch size of 32, enabling mini-batch gradient computation and parallelized loading for faster processing.
- **Transformation Pipeline:**
  A consistent transformation pipeline was applied to all images, including resizing, tensor conversion, and normalization before feeding them into the model.

6.2 Model Architecture and Training Process

- **Model selection:**
  The ResNet50 architecture was chosen for this project due to its strong feature extraction capabilities and proven performance on large-scale image recognition tasks.The model was pre-trained on ImageNet to leverage transfer learning.The final fully connected (FC) layer was replaced with a new linear layer matching the number of classes in the Caltech-101 dataset.
- **Transfer Learning Approach:**
  Transfer learning enables efficient training by reusing the learned convolutional filters of ResNet50 while fine-tuning only the higher layers for the new dataset.
  This reduces computational cost and improves generalization even with a limited dataset size.
- **Training Configuration:**Loss Function: Cross-Entropy Loss was used to measure classification error across multiple categories.

- **Optimizer:** Adam optimizer was selected for its adaptive learning rate capabilities, with a learning rate of 1e-4.

- **Batch Size:** 32 images per batch ensured a balance between speed and GPU memory efficiency.

- **Epochs**: The model was trained for 20 epochs to achieve stable convergence without overfitting.

- **Hardware Utilization:**
  The training process was accelerated using an NVIDIA GeForce RTX 3050 GPU (CUDA 12.7), ensuring efficient matrix computation and faster convergence.

- **Performance Monitoring:**
  Training and validation accuracy were calculated at the end of each epoch. The model parameters were saved whenever the validation accuracy improved, ensuring the best version was retained.

- **Early Stopping Mechanism:**
  To prevent overfitting, an early stopping strategy was implemented, training automatically halted if validation accuracy did not improve for 5 consecutive epochs.

- **Model Saving:**
  The best-performing model weights were saved,which served as the backbone for later feature extraction and image similarity computations.

6.3 Feature Extraction and Embedding Generation

Purpose: After fine-tuning the ResNet50 model, the next step involves extracting high-dimensional feature embeddings for all images in the dataset. These embeddings represent the semantic content of each image and serve as the foundation for similarity computation.

Steps:

- **Model Preparation:**The **fine-tuned ResNet50 model** trained for 20 epochs was loaded using the saved checkpoint .The final classification layer (fully connected layer) was removed, retaining only the convolutional feature extractor portion of the model.This allowed the network to output **2048-dimensional feature vectors** for each input image.

- **Feature Extraction Mode:** The model was switched to evaluation mode (model.eval()), ensuring parameters remained static during inference and dropout/batch normalization layers behaved deterministically.
- **Input Preprocessing:** Each image was resized to 224×224 pixels and normalized using the ImageNet mean and standard deviation values.The same preprocessing pipeline as used during training ensured consistent feature distribution across training and inference phases.
- **Batch Processing:** The images from the Caltech-101 dataset were processed in batches of 32 to optimize GPU memory utilization and speed up computation.
- **Embedding Extraction:** Each image was passed through the modified ResNet50 model.The output from the final average pooling layer (before classification) was flattened to form a feature embedding vector representing each image's visual characteristics.These embeddings capture both low-level (edges, colors) and high-level (shapes, objects) semantics.
- **Storage of Embeddings:** Extracted embeddings were concatenated into a single NumPy array and saved in .npy format (train_embeddings.npy). Corresponding image file paths were stored in train_image_paths.npy.Both files were organized under the directory embeddings_20epochs/ for future retrieval and similarity computation.
- **Hardware and Performance Considerations:** The extraction process utilized an NVIDIA GeForce RTX 3050 GPU for efficient computation.The entire dataset embeddings were generated with high throughput while maintaining numerical stability using PyTorch's no-gradient inference mechanism.

6.4 Similarity Computation and Image Retrieval

- **Objective of This Stage:** The goal of this module was to identify and retrieve images that are visually similar to a given query image.This was achieved by comparing the pre-computed embeddings of dataset images against the embedding of a user-supplied query image using a similarity metric.
- **Model Loading and Preparation**: The fine-tuned **ResNet50 model** (trained for 20 epochs) was reloaded from the checkpoint file resnet50_finetuned_20epochs.pth. Similar to the embedding generation stage, the **final fully connected classification layer** was removed, retaining only the convolutional base of the model.This modified model produced **feature vectors** instead of class probabilities, ensuring that the extracted representation captured semantic similarity.
- **Query Image Preprocessing:** The input query image was loaded and **converted to RGB** format for consistent processing.It was **resized to 224×224 pixels**, converted to a PyTorch tensor, and normalized using **ImageNet mean and standard deviation** values.This ensured uniformity between the query image and the dataset images used during model training and embedding generation.

- **Query Embedding Generation:** The query image tensor was passed through the feature extractor model in **inference mode** (using torch.no_grad() to disable gradient computation).The resulting output feature map was **flattened** to form a **2048-dimensional embedding vector** representing the query image in the learned feature space.

- **Loading Stored Dataset Embeddings:** Pre-computed embeddings (train_embeddings.npy) and their corresponding image paths (train_image_paths.npy) were loaded from storage.These embeddings represent the entire **Caltech-101 dataset**, which serves as the searchable database.

- **Similarity Computation:** The **cosine similarity metric** was used to measure the angular closeness between the query embedding and all stored embeddings.Cosine similarity values range from **-1 to +1**, where a value closer to **+1** indicates a higher degree of visual similarity between images.The similarity scores were sorted in descending order to retrieve the **Top-5 most similar images**.

- **Result Visualization:** The system displayed the query image alongside the top five retrieved images with their corresponding similarity scores. Each retrieved image reflected a **visually and semantically similar** representation to the query, validating the model's ability to capture underlying image features effectively. The results were visualized using **Matplotlib**, allowing direct comparison of retrieved outputs.

- **Interpretation:** Images showing similar patterns, textures, and shapes to the query were consistently ranked higher, confirming the robustness of the ResNet-based embedding representation.The approach successfully demonstrated **content-based image retrieval (CBIR)** capabilities using deep learning rather than traditional handcrafted features.

7. RESULT

7.1 Training Performance

- The model was trained for 20 epochs using the Adam optimizer with a learning rate of 1e-4.
- The **training and validation accuracy** improved steadily over the epochs, with early stopping ensuring that the model did not overfit.
- The **best validation accuracy** achieved was approximately **96%**, indicating strong generalization capability on unseen data.
- The saved model (resnet50_finetuned_20epochs.pth) served as a robust backbone for feature extraction during the similarity search phase.

7.2 Embedding Generation and Feature Representation

- Each image in the dataset was transformed into a 2048-dimensional embedding vector using the trained ResNet50 model.
- The embeddings captured high-level visual semantics such as shape, texture, and spatial patterns instead of low-level pixel details.
- The generated embeddings were stored as NumPy arrays (train_embeddings.npy) for fast retrieval and analysis.

7.3 Image Similarity and Retrieval Results

- During testing, the system successfully retrieved Top-5 most visually similar images to the given query image using cosine similarity.
- The retrieved results demonstrated semantic relevance — for instance, a query image of a car returned visually similar cars rather than unrelated objects.
- Similarity scores for the top matches typically ranged from 0.95 to 0.98, reflecting a strong visual correlation.
- The retrieval process was efficient, requiring only a few seconds per query due to pre-computed embeddings.

7.4 Observations:

- The training loss consistently decreased with epochs, reflecting stable convergence.
- Validation accuracy plateaued after around 12 epochs, triggering the early stopping mechanism.
- Images belonging to the same category (e.g., multiple dog breeds or vehicle types) produced embeddings that were closely clustered in feature space, validating the effectiveness of the learned representation.
- When queried with an image of a butterfly, the top retrieved images included other butterfly species with similar wing patterns and colors, confirming that the model learned fine-grained visual distinctions.

8. DISCUSSION

- The results validate the **effectiveness of transfer learning** in adapting pre-trained models (ResNet50) to specialized datasets like Caltech-101.

- The **cosine similarity metric** proved efficient for measuring visual correlation, maintaining both accuracy and computational efficiency.

- The **embedding-based retrieval approach** significantly outperformed traditional methods by capturing both low- and high-level visual cues.

- Minor mismatches in retrieval results were observed for classes with visually similar patterns (e.g., certain animal or object textures), which could be improved with **data augmentation or fine-grained classification** techniques.

9. CONCLUSION

This project presented an Image Similarity Search and Recommendation System built using a ResNet50 deep learning model trained on the Caltech-101 dataset. The system effectively extracts visual features from images and identifies similar ones using cosine similarity, achieving a best validation accuracy of 96%. The results clearly demonstrate that deep learning–based embeddings outperform traditional text or color-based search methods in understanding image content.

The implementation using PyTorch in VS Code proved efficient for feature extraction, training, and testing. The system shows great potential for real-world applications such as e-commerce product recommendations, digital media management, and content-based image retrieval. Overall, the project successfully achieves its goal of retrieving and recommending visually similar images with high accuracy

10. FUTURE SCOPE OF WORK

- **Fine-tuning with Larger and Diverse Datasets:**
  The current model is trained on the Caltech-101 dataset. Future work can involve fine-tuning on larger and more diverse datasets (e.g., ImageNet, CIFAR-100, COCO) to enhance generalization and robustness.

- **Dimensionality Reduction Techniques:**
  High-dimensional embeddings can be reduced using methods like **PCA**, **t-SNE**, or **UMAP** for efficient storage, visualization, and faster similarity search.

- **Integration with Retrieval Systems:**
  The extracted embeddings can be used in **image retrieval** or **content-based recommendation systems**, improving real-world applications such as product search, face recognition, and object clustering.

- **Experimenting with Other Architectures:**
  Future studies can compare the performance of ResNet50 embeddings with architectures like **Vision Transformers (ViT)**, **EfficientNet**, or **ConvNeXt** for better accuracy and computational efficiency.

- **Real-time Embedding Generation:**
  Implementing optimized inference pipelines using **ONNX**, **TensorRT**, or **PyTorch JIT** can allow real-time embedding extraction on edge devices or in production servers.

11. REFERENCES

[1] A. Babenko, A. Slesarev, A. Chigorin, and V. Lempitsky, "Neural Codes for Image Retrieval," *ECCV*, 2014. DOI: 10.1007/978-3-319-10590-1_53

[2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *ICLR*, 2015.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. DOI: 10.1109/CVPR.2016.90

[4] R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, and J. Sivic, "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," *CVPR*, 2018.

[5] Y. Zhang et al., "Deep Learning-Based Visual Recommendation System," *IEEE Access*, vol. 8, pp. 104223–104233, 2020.